# 通知抽取助手 · 技术文档

> 队伍：Triple Y ｜ 比赛：传智杯 AI WEB 网页开发挑战赛 版本：v1.4 ｜ 日期：2026-11-11（模型 ID 复核）
>
> 说明：本文由 A（李丹阳）与 C（方可圆）共同汇总。当前为省赛提交版；团队正式 Key 已配置，文本与视觉使用 DeepSeek Flash，首轮真实验收完成，DeepSeek 控制台累计 70 次请求 / 214,613 tokens / ¥0.61（2026-10-29）。

## 目录


1. 项目概述
2. 需求分析
3. 系统架构设计
4. AI 技术方案（4.1 模型选型 / 4.2 Prompt 设计 / 4.3 输出稳定性 / 4.4 多模态与 OCR / 4.5 RAG 与可解释性 / 4.6 缺项判断 / 4.7 Agent 与工具 / 4.8 通知变化 / 4.9 安全可控 / 4.10 评估优化）
5. 核心功能实现
6. 测试与部署
7. 创新点
8. 总结与展望 附录：运行命令、环境变量与测试数据


---

## 1. 项目概述

**问题背景**：校园与职场中的"办事通知"以纯文字、截图、PDF 等形式散落在群消息和公告里，关键信息（截止时间、材料清单、办理步骤、审核要求）分散在长文中。人工阅读容易漏材料、看错时间、混淆流程；通用大模型直接回答又容易编造，用户无法核验。

**一句话价值**：把一段通知变成**可勾选、可溯源、可提醒**的待办清单。

**目标用户**：在校学生（奖助学金、竞赛报名、证件办理）、社团/单位内勤（活动立项、经费报销、场地申请）、职场人士（入职材料、资质办理、行政审批）。

**本期范围（P1）**：文字通知解析、截图识别、文字型 PDF 提取、三档结构化输出、材料清单与缺项核对、查看依据、材料库检索与追问、补充通知对比与变更确认、清单版本/导出、日期核对、会话隔离与删除、清单保存与恢复。
**后续规划（P2）**：多文件合并解析、复杂条件分支、向量检索/重排、日历同步、多人协作、网页链接抓正文。

### 1.3 与通用 AI / 传统方案的区别

| 方案 | 输出 | 主要问题 |
| --- | --- | --- |
| 人工读通知 + 自己列清单 | 个人笔记 / Excel | 慢、容易漏材料、看错截止时间、无法追溯原文 |
| 通用大模型直接总结 | 一段自然语言 | 可能编造、无法勾选、无法核对缺项、无法保存为任务 |
| 固定规则模板 | 固定字段 | 泛化差，遇到新通知就要改规则，维护成本高 |
| **本项目** | **Matter JSON + 可勾选清单 + 原文出处 + 可调用工具** | 需要模型/OCR 能力，复杂表格仍有局限，但输出可执行、可核验、可降级 |

核心差异不是“总结得更漂亮”，而是把通知变成**能执行、能核对、能追溯**的结构化对象。

## 2. 需求分析

### 2.1 典型场景

- 学生收到一条奖学金通知：需要知道**截止时间**、要交**哪些材料**、**交给谁/交到哪**、**先做什么后做什么。**
- 社团收到立项通知：需要知道**审核环节**、**依赖关系**（审核通过后才能缴费）、**风险点**（时间紧、材料复用）。
- 用户手上已有部分材料：需要系统告诉他**还缺什么**，而不是重复罗列全部要求。

### 2.2 功能需求

| 编号 | 需求 | 验收标准 |
| --- | --- | --- |
| F1 | 文字通知 → Matter JSON | 返回 title / deadline / materials / step_list 等，结构符合 Schema |
| F2 | 三档复杂度自适应 | simple 不出现材料/步骤；standard 有材料与步骤；complex 增加审核/依赖/预警 |
| F3 | 用户情况核对 | 勾选材料 + 自述情况 → 已准备 / 缺少 / 需要确认 三类 |
| F4 | 查看依据 | 每条要求可查看原文片段；无依据标"系统建议" |
| F5 | 截图解析 | 上传截图 → 识别文字 → Matter；格式/大小受限 |
| F6 | PDF 解析 | 文字型 PDF → Matter；扫描版明确提示 |
| F7 | 材料库与追问 | 多材料上传、片段编号、检索、回答附出处、材料外拒答 |
| F8 | 保存与恢复 | 准备清单/修改内容保存后，刷新或重启仍可恢复 |

### 2.3 非功能需求

| 维度 | 要求 | 实现 |
| --- | --- | --- |
| 稳定性 | 模型不可用时服务不崩、前端有可读结果 | 视觉→本地 OCR→Mock 三层降级；统一错误信封 |
| 可解释 | 关键结论能追溯到原文 | `source_ref.quote` + `sources` + 片段编号 + `is_suggestion` |
| 性能 | 联调/演示响应可接受 | Mock 实测 100 次：平均 10.4ms、P95 16.9ms；真实模型 10~30s（已在前端设 90s 超时） |
| 安全性 | 密钥不入库、材料外不乱答、输出不注入 | `.env` + `.gitignore`；无依据拒答；前端 `esc()` 转义 |
| 可维护性 | 结构清晰、可自测 | 9 套自测脚本 + `preflight.py` 一键自检 + `check_sync.py` |

---

## 3. 系统架构设计

系统采用前后端分离的三层结构：

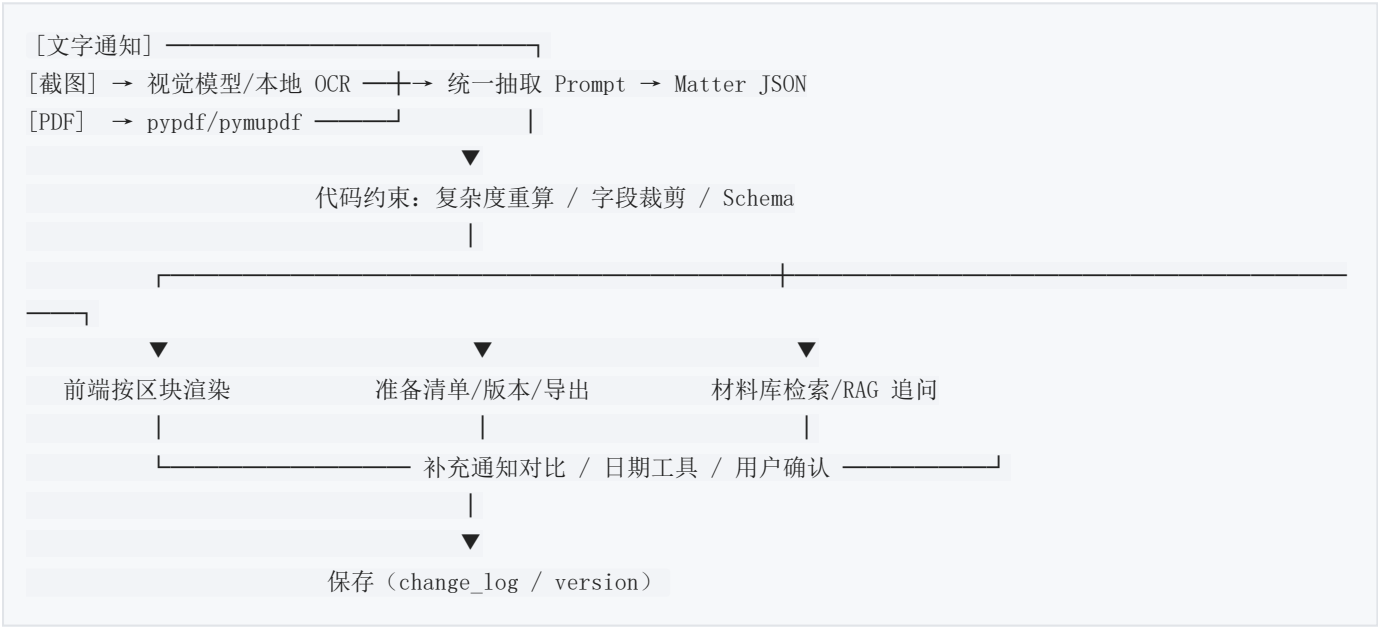
```
浏览器（前端 5173 / 静态托管）
        |  HTTP + JSON / multipart
        ▼
FastAPI 后端（8000）
  ├── 入口层：CORS、参数校验、统一响应、异常兜底
  ├── 能力层：文本抽取 / 图片解析 / PDF 提取 / 准备清单 / 查看依据 / 文件检索 / 追问
  |          / 双通知对比 / 版本与导出 / 日期核对 / 会话隔离
  ├── 抽取层：A 的 service.py（Prompt + LLM + Matter 结构 + Schema 校验）
  └── 存储层：backend/data（准备清单、文件片段索引、对比记录，JSON 原子写）
        |
        ▼
大模型（DeepSeek 官方 API，可配置）
  文本模型：DeepSeek        视觉模型：TEAM_LLM_MODEL（可选）
  无 Key 时：离线 Mock + 本地 OCR（RapidOCR）
```

核心设计原则：**模型只负责"读懂"，结构、判定与降级由代码兜底**，保证演示可复现、失败有反馈。

## 3.1 模块职责

| 模块 | 职责 | 关键技术 |
| --- | --- | --- |
| 前端 | 通知输入、结果卡片、材料勾选、任务清单、对比/确认、出处弹窗、导出 | HTML/CSS/JS + Vite，按 `enabled_sections` 渲染 |
| 后端入口 | CORS、参数校验、上传限制、统一响应、异常兜底、限流、会话隔离 | FastAPI + Uvicorn |
| AI 抽取层 | 文本/截图/PDF → Matter；Prompt、Schema、复杂度、RAG、对比、日期工具 | `service.py` + `notification_extractor.py` + `prompt.py` |
| 业务能力层 | 缺项核对、准备清单、版本历史、导出、材料库、对比记录 | `prepare_list.py`、`requirement_checker.py`、`compare_store.py` |
| 存储层 | 清单/文件索引/对比记录持久化 | 线程锁 + JSON 原子写；Docker volume |
| 模型层 | 文本生成、视觉识别、Function Calling | DeepSeek 文本 API；视觉模型可选；无 Key 时 Mock + RapidOCR |

## 3.2 三条输入通道与数据流




## 3.3 部署拓扑



本地开发用 `uvicorn` + `vite`；公网演示可用 Docker Compose 或 Render Blueprint，二者都保留同一套环境变量契约。

### 3.4 接口清单

| 接口 | 作用 |
| --- | --- |
| `POST /api/analyze` | 文字 → Matter |
| `POST /api/check` | 用户勾选 + 自述 → 缺项核对 |
| `POST /api/prepare-list` / `save` / `GET` | 准备清单生成 / 保存 / 读取 |
| `POST /api/evidence`、`/index` | 要求 → 原文依据（无依据标为系统建议） |
| `POST /api/analyze-image` | 截图 → Matter（视觉模型 / 本地 OCR） |
| `POST /api/analyze-pdf` | 文字型 PDF → Matter |
| `POST /api/files`、`GET /api/files`、`POST /api/files/search` | 材料上传、片段编号、检索 |
| `POST /api/ask` | 先检索再回答，附出处，材料外拒答 |
| `POST /api/compare`、`GET /api/compare`、`DELETE /api/compare/{id}` | 双通知对比、保留各自来源、按会话删除 |
| `GET /api/tools/check-date` | 日期核对（星期/距今/周末/常见节假日） |
| `GET /api/prepare-list/{id}/history`、`/export`、`DELETE` | 版本历史、CSV/Markdown 导出、删除 |
| `GET /health` | 服务状态与当前模式（mock/real） |

所有接口统一返回 `{success, data, error}`，且始终 HTTP 200，前端只需判断 `success`。

## 4. AI 技术方案

本节按“模型选择 → Prompt → 结构稳定 → 多模态 → RAG → 缺项判断 → Agent/工具 → 通知变化 → 安全 → 评估”展开。

| 小节 | 回答的问题 |
| --- | --- |
| 4.1 模型选型 | 为什么用通用模型、为什么文本/视觉分开、Key 与降级怎么做 |
| 4.2 Prompt 设计 | 怎么让模型稳定输出 Matter、怎么防编造 |
| 4.3 输出稳定性 | 模型输出漂移时，代码怎么兜底 |
| 4.4 多模态 | 截图/PDF 怎么进入同一套抽取 |
| 4.5 RAG | 材料库怎么检索、怎么附出处、怎么拒答 |
| 4.6 缺项判断 | 怎么从“通知要求”推到“用户还缺什么” |
| 4.7 Agent/工具 | 模型怎么调 `check_date`、为什么不是全自动 |
| 4.8 通知变化 | 补充通知怎么对比、确认、留档 |
| 4.9 安全可控 | 幻觉、注入、误操作、密钥、会话怎么防 |
| 4.10 评估优化 | 用什么指标评估、下一步怎么优化 |

# 4.1 模型选型

## 4.1.1 选型约束

本项目的输入是中文办事通知，输出是字段固定的 Matter JSON，并且要同时满足“可解释、可降级、可演示”。因此模型选型不是单纯比较“谁更聪明”，而是按下面 7 个约束综合取舍：

| 约束 | 具体要求 | 对选型的影响 |
| --- | --- | --- |
| 中文长文本 | 能稳定理解公文、通知、表格截图里的中文 | 优先选中文语料充分、中文指令跟随稳定的通用模型 |
| 结构化输出 | 能按指定 JSON 字段输出，不能夹带解释 | 需要支持 JSON 模式或强 JSON 指令跟随的模型 |
| 多模态 | 截图里可能同时有文字、表格、数字、日期 | 文本模型与视觉模型分开配置，视觉模型单独选型 |
| 时延与成本 | 单次解析 10–30 秒可接受，演示成本可控 | 不盲目追求最大参数模型；按任务难度选性价比档位 |
| 供应商可用性 | 比赛现场网络、额度、接口稳定性不可控 | 团队统一使用 DeepSeek 官方 API，账号/额度/账单集中管理 |
| 密钥安全 | Key 不能进入前端和公开仓库 | Key 只放后端 `.env`，前端只调后端 |
| 可降级 | 没有 Key 或接口失败时仍能演示 | 无 Key 走 Mock / 本地 OCR，接口失败有明确错误与兜底 |

## 4.1.2 文本模型：团队统一使用 DeepSeek

团队正式 Key 统一使用 DeepSeek 的文本模型，通过环境变量配置：

```
# 团队正式 Key 使用 DeepSeek
LLM_API_KEY=<your-deepseek-key>
LLM_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-flash
```

> 模型 ID 以 API `/models` 返回为准：团队 Key 返回 `deepseek-flash`、`deepseek-v4-pro`；实际调用 `deepseek-flash` 返回 `model=deepseek-flash`，`deepseek-chat` 只是兼容别名，不作为团队配置，也不把平台展示名称当作 API model ID。

选择 DeepSeek 的理由：

1. **团队账号统一**：一个团队 Key、一个文本模型，额度、账单和调用记录集中管理，避免多人用不同供应商/不同 Key 导致结果不可复现。
2. **中文通知理解合适**：DeepSeek 对中文公文、通知、材料清单这类长文本的指令跟随和语义理解比较稳定，适合“把通知拆成结构化字段”的任务。
3. **结构化输出可用**：使用 `response_format={"type": "json_object"}` + `temperature=0`，配合 JSON Schema 和代码字段裁剪，能稳定产出 Matter；默认选 `deepseek-flash` 做抽取，`deepseek-v4-pro` 不作为默认，因为抽取任务对延迟和成本更敏感。
4. **成本可控**：按量计费，输入长度受 `MAX_TEXT_LENGTH` 控制，抽取任务不需要长链推理；相比每次上最大参数模型，更适合比赛演示和反复联调。
5. **国内访问更稳定**：比赛现场网络、代理和跨境延迟不可控，使用国内可访问的 DeepSeek 官方 API，减少现场演示的网络风险。
6. **接入成本低**：DeepSeek 提供标准 Chat Completions 接口，项目现有 SDK 可以直接调用；`LLM_BASE_URL` / `LLM_MODEL` 保持配置化是为了本地测试和后续迁移，**团队正式环境固定使用 DeepSeek。**
7. **可降级**：没有团队 Key 时走离线 Mock；接口失败返回统一错误信封，不会让页面白屏。

**为什么不做微调 / 不用本地小模型?**

本项目的难点不是“模型知不知道什么是奖学金”，而是“能不能稳定输出指定结构、能不能不编造、能不能和代码规则配合”。微调需要标注数据、训练资源和持续维护，比赛周期内收益不稳定；本地 7B 级模型在中文长通知、JSON 稳定性、工具调用上还需要额外评测和调优。当前方案把“语义理解”交给 DeepSeek，把“结构、判定、降级”交给代码，整体更可控，也符合团队统一使用 DeepSeek 的部署方式。

### 4.1.3 视觉模型：单独配置，本地 OCR 兜底

截图解析使用独立的视觉模型配置：

```
# 当前团队 Key 支持图片输入，视觉模型复用同一个 DeepSeek Key
# 如果控制台实际模型名不同，只改 TEAM_LLM_MODEL
TEAM_LLM_API_KEY=<your-deepseek-key>
TEAM_LLM_BASE_URL=https://api.deepseek.com
TEAM_LLM_MODEL=deepseek-flash
```

> 当前团队 Key 支持图片输入，视觉模型复用同一个 DeepSeek Key；如果后续额度/模型调整，视觉模型可以单独替换。视觉调用失败时仍自动降级 RapidOCR，不影响文字主流程。

单独配置的原因：

- 视觉模型与文本模型解耦，便于单独换模型/额度；当前复用同一个 DeepSeek Key；
- 截图请求更贵、更慢，只在用户上传图片时调用，不拖累文本主流程；
- 视觉模型不可用时，自动降级到 **RapidOCR（onnxruntime）** 本地识别，再把文字送入统一抽取管道；

- 对“字迹模糊、表格错位、数字被切断”等视觉歧义，模型输出 `confirmation_needed`，让用户确认，而不是猜。

### 4.1.4 模型分工

| 任务 | 使用的模型/工具 | 输出 | 失败时 |
| --- | --- | --- | --- |
| 文字通知抽取 | 文本模型（`LLM_*`） | Matter JSON | 无 Key → Mock；接口失败 → 可读错误 |
| 截图识别 | 视觉模型（`TEAM_LLM_*`） | Matter JSON + `confirmation_needed` | 无 Key/失败 → RapidOCR → 统一抽取 |
| PDF 文字提取 | pypdf / pymupdf | 纯文本 + 页码 | 扫描件 → 明确提示改用截图 |
| 材料库追问 | 文本模型 + 本地检索 | 回答 + 出处 | 无 Key → 直接返回最相关原文；无依据 → 拒答 |
| 日期核对 | 本地 `check_date` 工具 | 剩余天数/星期/风险/建议 | 不依赖模型，始终可执行 |
| 补充通知对比 | 规则差异 + 可选文本模型 | `diff` + 受影响待办 | 模型失败 → 规则差异兜底 |
| 状态推演 | 文本模型 + 代码兜底 | 更新后的 Matter | 模型失败 → `_code_level_fallback` |

### 4.1.5 Key 配置、安全与降级

- Key 只从后端环境变量读取，前端不接触；`.env` 已被 `.gitignore` 忽略，仓库只提交 `.env.example`。
- 没有 Key 且 `LLM_ENABLE_MOCK=1` 时，`/health` 返回 `mode=mock`：文本走离线 Mock，截图走本地 OCR，保证演示不白屏。
- 配置团队 DeepSeek Key 后 `/health` 返回 `mode=real`，文本与 RAG 走真实模型；视觉模型已配置时截图走真实视觉模型，否则继续走本地 OCR。
- 开发阶段的个人 Key（非团队 Key）只用于本机联调，没有提交到仓库，也不作为团队验收数据。
- 真实模型接口失败时，后端返回统一错误信封，不会把异常堆栈或 Key 暴露给前端。
- 真实模式默认按 IP + 路径限流（普通接口 60 次/分钟、上传 10 次/分钟），避免演示额度被刷爆。

### 4.1.6 当前验收状态（重要）

截至本版提交，**团队 DeepSeek Key 已配置**，文本与视觉均使用 `deepseek-flash`，`/health` 返回 `mode=real`，`preflight.py` 13/13 通过。首轮真实验收已完成：

| 项目 | 当前状态 | 还差什么 |
| --- | --- | --- |
| 文本解析 | 3 个样本通过；单条约 20–42s；01/02 为 complex，03 修复后为 standard | 平台累计 70 次 / 214,613 tokens / ¥0.61；单场景未分摊 |
| 截图识别 | 真实视觉模型通过；约 9.6s；标题/材料/日期可识别；deadline 已归一化为 ISO | 无 |
| RAG 追问 | 材料内回答附出处；材料外明确拒答 | 纳入平台累计 |
| 日期工具调用 | 真实 Function Calling 成功，`tool_trace` 有 `check_date` | 无 |
| 补充通知对比 | A 侧规则+LLM 对比通过，改期/新增材料/受影响待办正确；C 侧 `/api/compare` 已改用增量合并，06+07 复测通过 | 已完成（smoke_test_hardening.py 19/19） |

> 结论：**当前按真实 DeepSeek Flash 演示**，不再标 Mock。平台累计 70 次 / 214,613 tokens / ¥0.61；C 侧 compare 增量误报已修复。完整记录见 真实模型验收记录.md 。

## 4.2 Prompt 设计

### 4.2.1 Prompt 的组织方式

Prompt 不是一段“万能提示词”，而是按任务拆成 6 个专用模板，统一放在 `src/prompt.py`，并同步到 `backend/prompt.py`：

| Prompt | 用途 | 关键约束 |
| --- | --- | --- |
| `SYSTEM_PROMPT` | 文字通知 → Matter | 只抽原文、禁止编造、三档模式、字段互斥、todo 非空 |
| `UPDATE_PROMPT` | 用户操作 → 状态推演 | 交付动作防误勾、顺序回填、保持 ID 不变 |
| `IMAGE_CHECK_RULES` | 截图识别附加规则 | 只对“看不清”写 `confirmation_needed`，不把缺失当模糊 |
| `RAG_QA_PROMPT` | 材料库追问 | 先查材料再回答、必须附出处、无依据拒答 |
| `COMPARE_PROMPT` | 原通知 vs 补充通知 | 找改期/增删材料/冲突，拿不准必须让用户确认 |
| `DATE_TOOL_SYSTEM_PROMPT` | 日期工具调用 | 必须先调用 `check_date`，再根据工具结果给建议 |

拆分的好处是：每个 Prompt 只负责一类判断，规则更短、更不容易互相冲突；出问题时也能定位到具体模板，而不是在一段超长 Prompt 里猜。

### 4.2.2 角色与任务边界

`SYSTEM_PROMPT` 的第一条不是“尽量提取”，而是**只提取原文出现过的内容**：

- 原文明确写出的字段，`is_suggestion=false`，必须带 `source_ref.quote`；
- AI 自己补充的建议，`is_suggestion=true`，`source_ref.quote` 可以为空；
- 原文没有的字段直接省略，禁止输出 `null` 占位；
- 原文只有“12月10日前”但没有年份，按缺失处理，不编造年份；

- “各学院需组织……”这类不属于个人用户的动作，不进入个人 `step_list`。

这条边界直接对应比赛的“AI 安全与可控性”：用户能看到每条要求的原文依据，没有依据的内容会被标成“系统建议”，而不是伪装成原文。

### 4.2.3 系统时间注入与相对时间归一化

Prompt 里注入 `NOW = {{NOW}}`，由服务端在每次请求时写入当前时间（Asia/Shanghai）。模型负责把“下周三前”“月底前”“通知发布后 7 天内”这类相对时间换算成绝对时间，统一输出 ISO 8601（+08:00）。代码层再做一次解析和校验，避免模型输出“下周三”这种前端无法计算的值。

### 4.2.4 三档模式：模型先判断，代码再重算

Prompt 要求模型输出 `detected_features`、`complexity_score` 和 `mode`，但**最终模式不采信模型**。服务端在 `notification_extractor.py` 里做三件事：

1. 用固定公式重算复杂度：

```
score = 0.2*has_deadline + 0.1*has_multiple_reminders + 0.2*has_multiple_steps
      + 0.2*has_materials + 0.1*has_review + 0.1*has_dependencies
      + 0.1*has_recurrence + 0.1*has_sub_steps
```

2. 按优先级判定：

```
has_sub_steps / has_review / has_dependencies → complex
score < 0.3 → simple
0.3 ≤ score ≤ 0.6 → standard
score > 0.6 → complex
```

3. 取“模型给的 mode”和“公式算出的 mode”中**更高**的一档，防止模型把复杂通知降级成 simple，导致材料/步骤被裁掉。

这样同一份通知在不同模型、不同温度下，模式判定仍然稳定、可复现。

### 4.2.5 字段互斥与 `enabled_sections`

三档模式不是“字段多一点少一点”，而是有明确的互斥表。`enforce_mode` 会直接删除当前模式不允许的字段：

| 模式 | 允许的 `enabled_sections` | 主要保留字段 | 明确禁止 |
|---|---|---|---|
| simple | summary, schedule, source | title, deadline, reminders, sources | materials, step_list, todo_list, warning_list, attachments, location 等 |
| standard | summary, schedule, content, steps, materials, source | 加 materials, step_list, attachments, key_points | recurrence, permissions, audit_log, comments |
| complex | 全部 8 个区块 | 全部字段 | 无 |

另外，simple/standard 的步骤里会删除 `sub_step_list`、`dependencies`、`parallel`、`condition`、`review`、`owner` 等复杂字段；standard 只保留 `deadline_risk`、`missing_material` 两类预警。这样前端只渲染

`enabled_sections` 声明的区块，不会出现“简单通知给一堆空模块”。

### 4.2.6 ID、命名与依赖规则

- `step_id` / `material_id` / `todo_id` 必须按顺序编号，例如 `step_1`、`mat_1`、`todo_1`；
- `material_name` 必须是 2–10 字的简短名词短语，例如“报名表”“成绩单”，不能把整句话塞进去；
- `step_list` 是“阶段”（2–5 个），`todo_list` 是“阶段里的具体动作”，两者粒度必须分开；
- 只有原文明确写了“A 完成后才能做 B”，才写 `dependencies`；没有明确依赖就按并行处理；
- `todo_list` 绝对不允许为空，`action` 必须是完整动宾短语，例如“准备并填写报名表”。

### 4.2.7 结构化输出的稳定性

Prompt 层和代码层共同保证 JSON 稳定：

1. **Prompt 层**：只输出 JSON、不要 Markdown、不要解释；字段名和类型写死；空数组/空对象省略；`temperature=0`。
2. **接口层**：优先使用 `response_format={"type": "json_object"}`；供应商不支持时自动去掉该参数重试。
3. **解析层**：去掉 ``` ```json 代码围栏后再 json.loads ```；解析失败返回可读错误，不把脏数据交给前端。
4. **校验层**：`jsonschema` 校验 Matter 结构，失败直接报错。
5. **修补层**：`_fix_todo_actions` 补空 action，`_fix_step_titles` 补空 title，`_rebuild_missing_items` 重建缺项，`_merge_ai_output` 以旧结构为基准合并，模型漏字段不会把原数据冲掉。
6. **兜底层**：模型失败时走 `_code_level_fallback`，至少保留基础状态联动。

### 4.2.8 状态推演 Prompt：防止“材料齐了就把提交动作也勾了”

`UPDATE_PROMPT` 的核心不是“让模型改状态”，而是给状态推演立规矩：

- **交付动作防误勾**：包含“提交/发送/上交/上传/打包/初审/审核/盖章”的动作，即使材料都准备好了，也不能自动标完成，必须用户明确确认；
- **顺序与并行区分**：有明确“先 A 后 B”的依赖，完成 B 时沿链回填 A；并行分支互不影响；
- **ID 不变**：禁止重命名或删除业务条目；
- **只改被提到的材料**：用户没说的材料保持原状，不猜；
- **代码兜底**：模型输出漂移时，`_sequential_backfill`、`_code_level_fallback`、`_fix_todo_actions` 继续完成基础联动。

### 4.2.9 图片 Prompt：只对“看不清”要求确认

`IMAGE_CHECK_RULES` 明确区分两类情况：

- **视觉歧义**（字迹模糊、纸张皱褶、遮挡、表格错位、数字被切断）→ 写入 `confirmation_needed`；
- **原文就没写**（只有“12月10日前”没有年份、单位没写全称、材料压根没提）→ 属于缺失，不写 `confirmation_needed`，也不编造。

这条规则避免把“信息缺失”误报成“识别失败”，也避免模型用确认弹窗掩盖自己没抽到的事实。

### 4.2.10 RAG / 对比 / 工具调用 Prompt

- **RAG 追问**：先检索、再回答；回答必须附“文件名 - 片段ID - 页码”；检索不到就输出“【未找到依据】……”，不允许用外部常识补答。
- **补充通知对比**：明确写“以此为准/原通知作废”时，改期按补充通知处理；没写时，改期、删材料一律要求用户确认；只输出差异，不重复编造原文。
- **日期工具调用**：要求模型先调用 `check_date`，再根据工具返回的 `days_left / is_weekend / suggested_finish_at` 生成建议；模型不调用时，代码层仍会执行一次工具，保证“至少有一次真实调用”。
- **状态更新**：只允许改状态和补缺项，不允许改业务字段；模型失败时保留原清单并返回错误。

### 4.2.11 Prompt 迭代经验

这版 Prompt 不是一次写成的，主要迭代点：

1. 从“让模型自由总结”改成“固定字段 + JSON Schema”，解决前端无法渲染的问题；
2. 从“模型自己给 mode”改成“公式重算 + 取更高档”，解决复杂通知被降级；
3. 从“材料/步骤混在一起”改成“step 是阶段、todo 是动作”，解决清单粒度混乱；
4. 从“没有依据也硬答”改成“source_ref + is_suggestion + 无依据拒答”，解决幻觉；
5. 从“改期直接覆盖”改成“差异预览 + 用户确认 + change_log”，解决补充通知误覆盖；
6. 从“日期让模型算”改成“模型调工具、代码执行”，解决模型算错日期和周末的问题。

## 4.3 输出稳定性（C 侧实现）

1. **复杂度重算**：不采信模型给的分数，用固定公式 `0.2·deadline + 0.1·多提醒 + 0.2·多步骤 + 0.2·材料 + 0.1·审核 + 0.1·依赖 + 0.1·周期 + 0.1·子步骤`，再按 `has_sub_steps/has_review/has_dependencies → complex`、`<0.3 simple`、`0.3–0.6 standard`、`>0.6 complex` 判定。
2. **字段裁剪**：`enforce_mode` 按模式删除互斥字段（simple 不出现 materials/step_list/todo_list 等），保证前端渲染契约稳定。
3. **Schema 校验**：`jsonschema` 校验 Matter 结构，失败直接返回可读错误，不把脏数据交给前端。
4. **三层降级**：真实模型 → 本地 OCR（图片）→ 离线 Mock（无 Key），任一层可用即可演示。

## 4.4 多模态与 OCR（A + C 共同实现）

### 4.4.1 分工与处理流水线

- **A 侧**：`analyze_notification_from_image` 视觉抽取函数、截图专用 Prompt 规则（表格/人数/日期/材料/金额）、`confirmation_needed` 判定。
- **C 侧**：`/api/analyze-image` 接口、文件大小/格式校验、本地 RapidOCR 兜底、统一抽取管道。

```
用户上传截图
  → 格式/大小校验（JPG/PNG/WEBP，≤5MB）
  → 有视觉 Key：视觉模型 + 截图专用 Prompt
  → 无 Key/调用失败：RapidOCR 识别文字 → 统一抽取 Prompt
  → Matter JSON + confirmation_needed
  → 前端黄色提示条 + 确认弹窗
```

### 4.4.2 截图识别重点

截图与纯文字不同，通知里的关键信息经常以“表格 + 短句 + 数字”混排出现。Prompt 要求优先识别：

| 类型 | 例子 | 输出字段 |
| --- | --- | --- |
| 时间 | 11月8日18:00、截止到本周五 | `deadline.due_at` |
| 材料 | 报名表、成绩单、身份证复印件 | `materials[]` |
| 人数/组队 | 每队 3–5 人、限报 2 队 | `key_points` / `warning_list` |
| 金额 | 报名费 50 元、资助 2000 元 | `key_points` |
| 表格 | 材料清单表、评审时间表 | `materials` / `step_list` |

### 4.4.3 视觉歧义与“看不清”规则

`IMAGE_CHECK_RULES` 只把**视觉歧义**写入 `confirmation_needed`：

- 字迹模糊、纸张皱褶、遮挡、反光；
- 表格错位、数字被切断、日期只有两位数年份；
- 同一字段出现两个互相冲突的值。

以下情况**不写** `confirmation_needed`，而是按“原文没写”处理：只写“12月10日前”没有年份、单位没写全称、材料压根没提。这样避免把“信息缺失”误报成“识别失败”，也避免模型用确认弹窗掩盖漏抽。

### 4.4.4 PDF 处理与边界

`POST /api/analyze-pdf` 接收 ≤10MB、≤30 页的文字型 PDF，用 pypdf / pymupdf 提取逐页文字，再走统一抽取。扫描版提取不到文字时明确返回“请改用清晰截图或直接粘贴文字”，不返回空 Matter。PDF 的页码会保留在片段元数据里，方便 RAG 引用时标注页码。

### 4.4.5 多模态融合策略

| 输入 | 预处理 | 抽取 | 失败兜底 |
| --- | --- | --- | --- |
| 文字 | 长度校验、`NOW` 注入 | 文本模型 | 无 Key → Mock |
| 截图 | 视觉模型 / RapidOCR | 文本模型（统一 Prompt） | 视觉失败 → OCR → Mock |
| PDF | pypdf / pymupdf 提取 | 文本模型（统一 Prompt） | 提取失败 → 提示改用截图 |

三条通道最终都收敛到同一个 Matter 结构和同一套代码约束，前端不需要为不同输入写三套渲染逻辑。

## 4.5 RAG 与可解释性（A 实现，C 接入）

### 4.5.1 RAG 架构

```
材料上传（TXT/MD/PDF）
  → 文本切片（按长度 + 重叠）
  → 片段编号（file_xxx_c1 / doc_pdf_1_p1_c1）
  → 检索打分（中文二元组 + 词频覆盖 + 连续字串 LCS）
  → Top-K 片段组装上下文
  → LLM 回答（必须附出处）
  → 无依据：直接拒答，不调用模型编造
```

A 侧负责 `NotificationRetriever` 检索器、打分、`ask_question_with_rag`、`source_ref` / `is_suggestion` 规则；C 侧负责材料上传/存储、`/api/files`、`/api/ask`、`/api/evidence` 接口接入。

### 4.5.2 切片、编号与检索打分

- 片段带唯一编号、文件名、页码（如有）、`char_start/char_end`；
- 检索前先做中文二元组、英文/数字词、停用词过滤；
- 打分综合三部分：词项覆盖率、词项权重、连续字串匹配（LCS）加分，低于阈值不返回；
- 返回 Top-K（默认 3–5）片段，前端展示文件名、片段号、匹配度。

> 当前是**词法检索**，不是向量检索。优点是零额外服务、可解释、演示稳定；缺点是同义改写、跨段落归纳召回弱，已列入 P2 的向量检索 + 重排。

### 4.5.3 生成、出处与拒答

`RAG_QA_PROMPT` 强制：

1. 回答 100% 来自检索片段，不使用外部常识；
2. 每条结论附 `【出处：文件名 - 片段ID（第X页）】`；
3. 检索为空时输出“【未找到依据】现有材料中未提及……”，不调用模型补答；
4. 多项要求分点回答，简练易读。

### 4.5.4 可解释性：source_ref 与系统建议

Matter 里每条材料/步骤/待办/截止/预警/要点都可以带 `source_ref.quote`。`/api/evidence` 返回原文依据；没有原文引用的条目标 `is_suggestion=true`，前端用黄色“系统建议（无原文依据）”展示。这样用户能区分“通知明确要求”和“AI 补充建议”，而不是把所有内容混成一段看似权威的总结。

### 4.5.5 评估指标与当前局限

| 指标 | 定义 | 当前状态 |
| --- | --- | --- |
| 字段抽取准确率 | 标题/截止/材料/步骤与原文一致的比例 | 真实 3 样本结构通过；字段级准确率待人工逐项核对 |
| 材料召回率 | 原文要求材料被抽出的比例 | 3 个样本均抽出 3 项材料；真实召回率待人工核对 |
| 出处命中率 | 回答引用的片段确实包含依据的比例 | 真实 RAG 3.5s 附出处；出处命中率待批量统计 |
| 拒答正确率 | 材料外问题被正确拒绝的比例 | 材料外“食堂”问题已拒答 |
| 检索耗时 | Top-K 检索耗时 | Mock 100 次平均约 10–14ms；真实 RAG 3.5s（含生成） |
| 生成耗时/费用 | 真实模型首 token/总耗时/token 费用 | 文本 20–42s、截图 9.6s、PDF 21.6s、工具 3.2s；平台累计 214,613 tokens / ¥0.61 |

后续优化：向量检索 + rerank、片段去重、按标题/材料类型加权、引用片段可点击回原文。

## 4.6 缺项判断与状态推演（A 规则 + C 接口）

### 4.6.1 三类核对结果

`/api/check` 的输入是 Matter + 用户勾选 + 用户自述，输出不是简单的“缺/不缺”，而是：

| 类别 | 含义 | 例子 |
| --- | --- | --- |
| `missing_required` | 原文明确要求、用户未勾选的必交材料 | 成绩单、身份证复印件 |
| `missing_optional` | 原文标为可选、用户未勾选 | 获奖证明（选填） |
| `unchecked_unspecified` / `need_confirm` | 原文没写清是否必交，或截止时间缺失 | “材料是否必交”“截止时间待确认” |
| `profile_issues` | 用户自述情况与原文要求冲突 | 原文要视频，用户说“没有” |

同时输出 `ready`、`ready_ratio` 和一句话结论，前端直接展示“还缺 2 项必交材料：成绩单、身份证复印件”。

### 4.6.2 状态推演规则

用户勾选材料或完成某一步后，系统不是简单地把所有相关项都打勾，而是按下面的规则推演：

1. **材料 → 待办联动**：材料 `prepared` 后，只把非交付类、且依赖该材料的待办置为 `completed`；
2. **交付动作防误勾**：包含“提交/发送/上交/上传/打包/初审/审核/盖章”的动作，即使材料齐了也不自动完成，必须用户明确确认；
3. **依赖链回填**：若后续步骤完成，且它与前置步骤是明确的顺序依赖，则沿链把前置材料标记为已准备；并行分支互不影响；
4. **代码层兜底**：模型输出漂移时，`_merge_ai_output` 以原结构为基准只吸收状态字段，`_fix_todo_actions` / `_fix_step_titles` 补空字段，`_code_level_fallback` 完成基础联动；
5. **缺项重建**：每次状态变化后重新计算 `missing_items`，只保留仍缺失的必填材料。

### 4.6.3 为什么不让模型直接改整份清单

模型适合判断“这句话是什么意思”，不适合保证“ID 不变、顺序不乱、旧字段不丢”。因此状态更新采用“模型只改状态、代码保留结构”的方式：模型返回的 `materials/step_list/todo_list` 只吸收 `status` / `status_desc`，其余字段以原 Matter 为准。即使模型漏返回某个列表，用户已保存的清单也不会被清空。

## 4.7 Agent 工作流与工具调用（A 实现）

### 4.7.1 受控 Agent，而不是全自动 Agent

本项目采用**受控 Agent 工作流**：

- 模型负责理解意图、选择是否调用工具；
- 代码负责执行工具、校验参数、兜底；
- 用户在关键动作（覆盖清单、删除材料、保存新版本）上确认。

这样做的原因是办事场景对“误操作”的容忍度很低：自动帮用户删掉材料、改掉截止时间，比“多问一句”风险大得多。我们把自主性限制在“信息整理”和“建议”层，把写操作交给用户确认。

### 4.7.2 工具清单

| 工具/能力 | 触发场景 | 输入 | 输出 |
| --- | --- | --- | --- |
| `check_date` | 清单有截止时间，需要核对星期/距今/周末 | `due_at`、可选 `now` | 剩余天数、星期、是否周末、风险等级、建议完成时间 |
| 补充通知对比 | 用户上传同一事项的原通知 + 补充通知 | 两份 Matter + 原文 | `changes`、`affected_todos`、`confirmation_needed`、`preview_todo_list` |
| 查看依据 | 用户点击某条要求的“出处” | Matter + 目标类型/ID | 原文片段；无依据标系统建议 |
| 准备清单 | 解析完成后生成待办 | Matter + 已勾选材料 | `todo_list`、`missing_items`、`summary` |

### 4.7.3 Function Calling 执行流程

```
用户清单/操作
  → 判断是否需要工具（有截止时间/有补充通知）
  → 模型返回 tool_calls（name + arguments）
  → 代码执行 check_date / compare
  → 工具结果以 role=tool 回传模型
  → 模型生成建议
  → 写入 date_check / suggestions / tool_trace
  → 前端展示“真实调用结果”
```

没有 Key 时不会假装“模型调用了工具”，而是走同一套工具执行轨迹：真实执行 `check_date`，返回 `tool_calls` 和 `date_check`，只是不做模型总结。这样演示的是“工具真的执行了”，不是“模型说它调用了”。

### 4.7.4 为什么不用 MCP

比赛评分里 MCP / Function Calling 是同一类“工具集成”能力。本项目选择 **Function Calling**，原因是：

- 部署简单：不需要额外 MCP Server、进程和协议适配；
- 工具数量少（日期、对比、依据），Function Calling 足够；
- 本地可降级：无 Key 时仍能执行工具；
- 后续可扩展：把 `check_date` 换成 MCP Server 只需替换工具执行层，不影响 Prompt 和前端。

### 4.7.5 失败与可观测性

- 工具参数非法：返回统一错误，不把异常抛给前端；
- 模型不调用工具：代码层补一次 `check_date`，保证至少有一次真实执行；
- 模型调用未知工具：返回“未知工具”，不执行；
- 每次调用记录 `tool_trace`（工具名、参数、结果），前端/答辩可展示真实轨迹。

## 4.8 补充通知变化处理（A 实现）

### 4.8.1 差异类型

两份通知分别解析为 Matter A / B，对比：

| 类型 | 判定 | 输出 |
| --- | --- | --- |
| 改期 | 两边都有 `due_at` 且不同 | `changed` + 旧/新时间 |
| 新增材料 | B 有、A 没有（名称归一化后） | `added` + 新材料 |
| 删除材料 | A 有、B 没有 | `removed`；未明确“不再要求”时必须确认 |
| 材料类型变化 | 必填/可选发生变化 | `changed` |
| 新增/删除步骤 | 步骤标题归一化后对比 | `added` / `removed` |
| 补充通知没提 | 某一类字段在 B 中缺失 | `info`，保留原值，不误删 |

### 4.8.2 什么时候必须让用户确认

- 补充通知没有写“以此为准 / 以本通知为准 / 原通知作废 / 替代原通知”，但截止时间冲突；
- 补充通知没有明确写“取消 / 不再要求 / 无需提交”，但原材料不再出现；
- 两份通知标题/事项明显不一致（人工判断）。

明确写了“以此为准”时，改期不再重复问“哪份有效”；但保存新清单前仍要用户点一次“确认更新”，这是写操作确认，不是歧义确认。

### 4.8.3 受影响待办与版本

```
原 Matter A + 补充 Matter B
  → diff（截止/材料/步骤）
  → affected_todos（新增/删除/更新截止/保留）
  → preview_todo_list（只预览，不写）
  → 用户确认
  → apply：更新 Matter + change_log.before + version+1
  → 未确认：applied=false，原清单不变
```

`change_log.before` 保留修改前的 `deadline/materials/step_list/todo_list`，支持回看；准备清单每次确认保存后版本递增，保留最近 5 版历史。

## 4.9 AI 安全与可控性

| 风险 | 具体表现 | 措施 |
| --- | --- | --- |
| 幻觉 | 编造材料、截止时间、办理地点 | `source_ref` + `is_suggestion` + 无依据拒答 + Schema 校验 |
| 提示词注入 | 通知正文里写“忽略以上规则” | System Prompt 优先、只输出 JSON、代码层字段裁剪，不执行正文指令 |
| 误操作 | 自动删材料、覆盖用户清单 | 写操作必须用户确认；未确认 `applied=false`；保留 `change_log` |
| 密钥泄露 | Key 进入前端/仓库 | Key 只在后端 `.env`；`.gitignore`；前端不接触 |
| 文件滥用 | 超大文件、非法格式 | 图片 ≤5MB、PDF ≤10MB/≤30页、材料库扩展名白名单 |
| 跨会话串数据 | 两个用户互相看到文件/清单 | `X-Session-Id` 隔离；删除接口；不对外提供跨会话查询 |
| 模型/网络故障 | 页面白屏、请求无响应 | 视觉→OCR→Mock 降级；统一错误信封；前端重试/提示 |
| 输出注入 | 模型输出 HTML/脚本 | 前端 `esc()` 转义；只渲染白名单字段 |

## 4.10 效果评估与优化策略

### 4.10.1 评估方法

- **字段级**：标题、截止时间、材料、步骤、待办分别比对原文，统计准确率/召回率；
- **引用级**：回答中的每条依据是否真的出现在对应片段；
- **拒答级**：材料外问题是否被正确拒绝；
- **工程级**：P50/P95 耗时、首 token 时间、错误率、降级触发率；
- **成本级**：prompt/completion token、单次解析费用、100 次演示预算。

### 4.10.2 当前基线

| 项目 | 结果 |
| --- | --- |
| 9 套后端自测 | 全部通过（C 侧记录） |
| A 侧 95 项确定性检查 | 95/95 |
| Mock 性能（100 次） | 平均约 10–14ms，P95 约 17ms |
| 真实 DeepSeek Flash 文本 | 3 个样本通过；单条约 20–42s |
| 真实视觉截图 | 通过；约 9.6s；2 材料；日期已 ISO |
| 真实 PDF | 通过；约 21.6s |
| 真实 Function Calling | 通过；1 次 `check_date`，约 3.2s |
| token / 费用 | 70 次 / 214,613 tokens / ¥0.61（控制台累计） |

### 4.10.3 优化策略

1. **Prompt 消融**：对比“只给字段说明”和“字段说明 + few-shot + 反例”，看 Schema 通过率和字段准确率；
2. **模型对比**：同一批通知比较 DeepSeek / 通义 / Moonshot 的 JSON 稳定性、中文抽取、价格；
3. **检索参数**：调整 chunk_size / overlap / top_k / 阈值，观察出处命中率与拒答率；
4. **规则兜底**：统计模型漏字段/空 action 的比例，决定是否增加代码修补；
5. **降级演练**：断 Key、断网、坏图、扫描 PDF 各跑一次，确保不白屏。

## 5. 核心功能实现（后端）

### 5.1 统一入口与统一管道

- `main.py` 负责 CORS、参数校验（空文本 / 超长 10000 字）、multipart 大小与格式校验、统一 `{success,data,error}` 响应和异常兜底；
- `analysis_pipeline.analyze_with_fallback` 负责“真实模型 / 离线 Mock”切换，文本、PDF、图片三条入口共用；
- 所有接口始终 HTTP 200，前端只判断 `success`，避免 422/500 直接把页面打崩。

### 5.2 文字通知 → Matter

输入一段通知文字，先注入 `NOW`，走 System Prompt + JSON Schema；输出 `title/deadline/materials/step_list/todo_list/sources`，再经过复杂度重算、字段裁剪、空字段修补和 `missing_items` 重建。失败时返回统一错误，不返回半截 JSON。

### 5.3 缺项核对与准备清单

用户勾选材料、填写自述后，`/api/check` 做确定性差集，输出 `missing_required / missing_optional / unchecked_unspecified / need_confirm / profile_issues`。`/api/prepare-list` 把 Matter 转成可执行待办，支持保存、读取、修改、版本历史和导出。每次确认保存 `version` 递增，`previous_todo_list` 与最近 5 版 `history` 留档。

### 5.4 查看依据

`/api/evidence` 接收 Matter + 目标类型/ID，返回 `source_ref.quote`；没有引用的条目标 `is_suggestion=true`。前端以弹窗展示原文，黄色标签区分“系统建议”。这让“模型说的”与“通知原文写的”在界面上可区分。

### 5.5 材料库与 RAG 追问

`/api/files` 上传 TXT/MD/PDF，切片并编号；`/api/files/search` 做词法检索；`/api/ask` 先检索再回答，附文件名、片段号、页码；检索为空时明确拒答。无 Key 时直接返回最相关原文与出处，不让页面卡死。

### 5.6 补充通知对比与确认

`/api/compare` 分别解析两份通知并保留文件名/原文/Matter，返回截止、材料、步骤差异和受影响待办。前端展示原要求/新要求/变化；未确认时只预览，确认后才写入新清单，并保留 `change_log.before`。

### 5.7 日期核对与导出

`/api/tools/check-date` 返回星期、距今、周末、常见法定节假日与“是否工作日”，调休提示人工确认；`/api/prepare-list/{id}/export` 支持 CSV（UTF-8 BOM，Excel 打开不乱码）与 Markdown。日期工具也被 AI 以 Function Calling 方式调用，结果写入 `date_check` / `suggestions` / `tool_trace`。

### 5.8 会话隔离、限流与删除

前端通过 `X-Session-Id` 区分浏览器会话，文件库、准备清单、通知对比互不可见；三类数据均提供删除接口。真实模型模式默认按 IP + 路径限流（普通 60 次/分钟、上传 10 次/分钟），Key 只从环境变量读取，不进入仓库。

## 6. 测试与部署

### 6.1 测试策略

| 层级 | 目标 | 方式 |
| --- | --- | --- |
| 确定性单元测试 | 规则、模式、日期、对比、检索、Prompt 契约 | `src/test_compare_date.py`（95 项，无 Key） |
| 接口集成测试 | 参数校验、错误信封、上传限制、会话隔离、限流 | `backend/smoke_test_*.py`（9 套） |
| 端到端验收 | 三档模式、截图、PDF、缺项、依据、追问 | `smoke_test_acceptance.py` + 演示数据包 |
| 演示前自检 | 健康检查到核心链路一键跑通 | `backend/preflight.py`（13 项） |
| 真实模型评测 | 字段准确率、引用准确率、耗时、费用 | 已首轮；字段级准确率待人工核对 |

## 6.2 自动化测试结果

| 脚本 | 项数 | 覆盖 |
| --- | --- | --- |
| `smoke_test.py` | 10 | 健康检查、正常/空/超长文本、非法 JSON、CORS、Mock 性能 |
| `smoke_test_check.py` | 10 | 缺项差集、全勾选、自述冲突、参数兜底 |
| `smoke_test_prepare.py` | 11 | 生成、保存、读取、修改内容持久化 |
| `smoke_test_evidence.py` | 8 | 索引、单条依据、无依据 → 建议、参数校验 |
| `smoke_test_image.py` | 6 | limits、合法图、格式/大小/空文件 |
| `smoke_test_pdf.py` | 6 | 文字型 PDF、扫描件、页数/大小限制 |
| `smoke_test_files.py` | 10 | 上传、片段编号、检索、追问、材料外拒答 |
| `smoke_test_acceptance.py` | 3 样本 | 三档模式端到端验收 |
| `smoke_test_hardening.py` | 19 | 双通知、版本历史、导出、日期、会话隔离、删除 |
| `preflight.py` | 13 | 演示前一键自检（含 10/28–10/30 新链路） |
| `src/test_compare_date.py` | 95 | 补充通知、日期、模式裁剪、检索、Prompt 契约（A 侧） |

> 当前结果包含真实 DeepSeek Flash 首轮验收；控制台累计 70 次 / 214,613 tokens / ¥0.61；C 侧 compare 增量误报已修复。

## 6.3 真实模型评测计划

| 维度 | 记录项 | 当前结果 |
| --- | --- | --- |
| 文本解析 | 3 个样本的字段准确率、耗时、prompt/completion tokens、费用 | 3 个样本通过；20–42s；平台累计 214,613 tokens / ¥0.61 |
| 视觉识别 | 截图材料/日期/金额识别率、`confirmation_needed` 命中率 | 通过；9.6s；2 材料；日期已 ISO |
| RAG 追问 | 出处命中率、拒答正确率、回答耗时 | 材料内 3.5s 附出处；材料外 0.03s 拒答 |
| 工具调用 | `check_date` 是否被模型调用、参数正确率、轨迹完整性 | 真实调用成功；1 次，3.2s |
| 补充通知 | 改期/增删材料/受影响待办 | A 侧通过；C 侧增量合并 19/19 通过 |
| 降级 | 断 Key / 断网 / 坏图 / 扫描 PDF 的页面表现 | 演示前再演练 |

## 6.4 部署

- 本地：`uvicorn` + `vite`，Mock 模式无需 Key；
- Docker：`backend/Dockerfile`（含 pypdf、RapidOCR）+ `docker-compose.yml`，`backend/data` volume 持久化，内置 HEALTHCHECK；
- 反向代理：Nginx + HTTPS，`proxy_read_timeout 180s`、`client_max_body_size 12m`；
- 可选公网：Render Blueprint（`render.yaml`），免费实例冷启动/临时磁盘需注意；

- 详见 `docs/部署说明.md`。

### 6.5 安全与运维

- Key 只在后端环境变量，`.env` 不入库；
- 真实模式默认限流（普通 60 次/分钟、上传 10 次/分钟），Mock 模式默认不拦；
- 文件、清单、对比记录按 `X-Session-Id` 隔离，支持删除；
- 统一错误信封，不把堆栈/Key 返回前端；
- 演示前跑 `preflight.py`，检查 `/health` 的 `mock/real` 状态。

---

## 7. 创新点

### 7.1 技术创新

1. **公式化复杂度判定 + 字段裁剪**：不让模型"凭感觉"给分，服务端用固定公式重算 complexity_score，并按模式强制裁剪字段，保证同一份通知的输出稳定、可复现。
2. **三档结构自适应**：simple / standard / complex 按需输出，前端只渲染 `enabled_sections` 中声明的区块，避免"简单通知给一堆空模块"。
3. **全链路降级**：视觉大模型不可用 → 本地 OCR（RapidOCR）→ 离线 Mock；文本模型不可用 → 离线抽取。演示与前端联调不会因为 Key/网络中断而白屏。
4. **可解释与防幻觉**：字段级 `source_ref` + "查看依据"接口；无原文依据的条目标为"系统建议"；材料库检索不到时直接回答"未找到依据"，不编造。
5. **受控 Agent + Function Calling**：模型负责选择工具，代码执行 `check_date`/对比，用户确认写操作；工具轨迹可展示，避免全自动 Agent 的误操作风险。
6. **补充通知的“预览—确认—留档”**：差异先预览，改期/删材料默认确认，确认后写新版本并保留 `change_log.before`，解决“通知改版把旧清单冲掉”的痛点。

### 7.2 应用创新

1. **从"读通知"到"能执行"**：输出不是一段摘要，而是可勾选的材料清单、可排序的步骤、可提醒的 DDL。
2. **用户情况驱动的缺项核对**：把"通知要求"和"用户已有"做差集，直接告诉用户还缺什么、什么需要确认。
3. **材料库 + 追问**：上传多份材料后可以问"视频有什么要求"，答案附文件名与片段编号，形成"办事资料助手"。
4. **通知改版管理**：同一事项多版通知不需要用户自己对照，系统直接列出改期、新增材料、受影响待办。

### 7.3 交互创新

- **出处弹窗**：点击材料/步骤/待办后的“出处”，直接看原文片段；没有依据的标黄色“系统建议”，不把 AI 补充伪装成原文。
- **差异预览**：补充通知先展示“原要求 / 新要求 / 变化 / 受影响待办”，用户确认后才覆盖，避免“静默修改”。
- **日期卡片 + 工具轨迹**：显示剩余天数、星期、周末/节假日提示，并能在答辩中展示 `tool_trace`，证明工具真实执行。
- **材料库追问**：回答下方展示文件名、片段编号、匹配度，用户可以判断“这句话是从哪份材料来的”。

### 7.4 与普通 AI 总结的对比

| 维度 | 普通 AI 总结 | 本项目 |
| --- | --- | --- |
| 输出形态 | 一段自然语言 | 结构化 Matter + 可勾选清单 + 待办 |
| 可核验 | 难以确认哪句话来自原文 | 每条要求带 `source_ref`，可点开看依据 |
| 可执行 | 用户还要自己整理 | 材料勾选、缺项核对、DDL、导出 |
| 多模态 | 多数只支持文字 | 文字 + 截图 + 文字型 PDF |
| 通知改版 | 重新总结，旧清单丢失 | 差异对比 + 用户确认 + 版本历史 |
| 失败表现 | 模型挂了就空白 | 视觉→OCR→Mock 降级，错误可读 |
| 工具能力 | 只能聊天 | Function Calling 调日期工具，结果回写 |

> 同一段“校园创新挑战赛”通知的实测对比（豆包 / DeepSeek / Kimi / 不漏办）见 对比-普通大模型-vs-不漏办.md。对比中暴露的条件材料、选填识别和最终步骤顺序已修复并复测；日期矛盾（下周三 vs 11/5 周四）仍列为 P1，不当作已完成能力宣传。

## 8. 总结与展望

### 8.1 已完成

文字/截图/PDF 三种输入通道；三档结构化输出；材料清单与缺项核对；条件材料（大一/大二及以上）与选填识别；最终步骤顺序与依赖；查看依据；材料库片段检索与追问；准备清单保存与恢复；双通知对比与变更确认；清单版本历史与 CSV/Markdown 导出；日期核对工具；会话隔离与删除；9 套自动化测试 + A 侧 95 项确定性检查 + 一键自检；Docker 部署方案与演示素材。

### 8.2 当前边界与后续规划

团队 DeepSeek Flash 已完成首轮真实验收（`preflight.py` 13/13、加固 19/19、结构验收 3/3、A 侧 95/95），控制台累计 70 次 / 214,613 tokens / ¥0.61。当前版本仍保持清晰边界：

- **文档类型**：聚焦清晰截图和文字型 PDF；扫描版 PDF、复杂表格、手写体、印章遮挡需要人工核对或改用截图通道。后续补扫描件 OCR 与表格结构识别。
- **检索方式**：当前是轻量词法检索（中文二元组 + 连续字串 + 词频），可解释、零额外服务；同义改写和跨段落归纳的召回仍有提升空间。后续加入向量检索与重排。
- **条件与复杂分支**：已支持“大一 / 大二及以上”条件材料和年级过滤；多层嵌套条件、多文件合并仍在路线图中，不把当前能力描述为“全自动理解所有通知”。
- **通知变化**：已支持增量合并、受影响待办、用户确认和版本留档；连续多份补充通知、跨段落语义改期需要更多真实样例验证。
- **日期与校历**：覆盖周末和常见固定节假日；农历节日、地方假期、调休以学校/单位通知为准，后续接可配置校历。
- **会话与权限**：`X-Session-Id` 只做浏览器会话隔离，不是登录鉴权；账号体系、权限管理、多人协作属于后续规划。

- **提醒与部署**：已生成 DDL 提醒任务，系统推送未接入；Docker / Render / VPS 配置已准备。在线演示为可选加分项，若服务器资源落实则补充公网地址，否则以本地演示和视频为准；演示视频按 5–8 分钟要求录制。

后续方向：向量检索与重排、复杂条件分支、多文件合并、日历/飞书/系统通知、多人协作、移动端体验与更完整的真实样本评测。

### 8.3 社会价值与商业潜力

**社会价值**：

- 降低“办事信息差”：学生、老年人、第一次办事的人不需要反复研究公文，减少漏交材料、错过截止时间；
- 减少返工：缺项核对和原文依据让用户一次准备齐，不用来回问辅导员/ HR；
- 可解释 AI：每条结论能回到原文，减少“AI 说的对不对”的信任成本。

**商业潜力**：

- **B2C 订阅/按次**：个人用户免费基础解析，高级功能（批量材料库、导出、提醒）订阅；
- **B2B SaaS**：学校学工/教务、企业 HR/行政、社团管理平台，按坐席或调用量收费；
- **私有化/API**：把“通知→Matter+清单”能力封装成 API，接入现有的办事大厅/OA；
- **成本优势**：支持 DeepSeek 等性价比模型，且无 Key 时可用 Mock/本地 OCR 降级，部署成本可控；
- **数据边界**：不接校务/政务系统、不自动提交表单，降低合规风险，适合先从“信息整理工具”切入。

### 8.4 展望

多文件合并解析、复杂条件分支（“若为应届生需额外提交…”）、向量检索与重排、日历/飞书同步、多人协作清单、移动端 App。

---

## 附录

### A.1 运行命令

```
 cd backend
.\.venv\Scripts\python.exe -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
cd ..\frontend
npm run dev

# 重新生成技术文档 HTML + PDF（A 修改 4.1/4.2 后运行）
powershell -ExecutionPolicy Bypass -File ..\scripts\build_tech_pdf.ps1
```

## A.2 环境变量

| 变量 | 说明 |
| --- | --- |
| `LLM_ENABLE_MOCK` | 无 Key 时是否走离线 Mock |
| `LLM_API_KEY` / `LLM_BASE_URL` / `LLM_MODEL` | 文本模型 |
| `TEAM_LLM_API_KEY` / `TEAM_LLM_BASE_URL` / `TEAM_LLM_MODEL` | 视觉模型（截图） |
| `MAX_TEXT_LENGTH` | 文本长度上限（默认 3000，演示配 10000） |
| `RATE_LIMIT_PER_MINUTE` | 普通模型接口每分钟限额（默认 60） |
| `UPLOAD_RATE_LIMIT_PER_MINUTE` | 上传接口每分钟限额（默认 10） |
| `RATE_LIMIT_ENFORCE_IN_MOCK` | Mock 模式是否强制限流（默认 0） |
| `X-Session-Id` | 前端请求头，用于用户数据隔离（不是登录鉴权） |

## A.3 测试结果

| 脚本 | 结果 |
| --- | --- |
| 9 套自测（接口/缺项/清单/依据/图片/PDF/文件/追问/加固） | 全部通过 |
| A 侧 `src/test_compare_date.py` | 95/95（补充通知/日期/模式/检索/Prompt 契约） |
| `smoke_test_acceptance.py` | simple / standard / complex 3/3 |
| `preflight.py` | 13/13，可开始演示 |
| `smoke_test_hardening.py` | 19/19 |
| `smoke_test_perf.py`（Mock 100 次） | 平均约 10–14ms，P95 约 17ms |
| `docs/技术文档.pdf` | 已重新导出 25 页最终版（≤30 页），含 4.1/4.2 |

## A.4 评分维度与文档对应

| 评分维度 | 权重 | 本文对应章节 | 备注 |
| --- | --- | --- | --- |
| AI 技术深度 | 35% | 4.1–4.10、6.2、7.1 | 模型选型、Prompt、RAG、多模态、缺项判断、工具调用、安全、评估 |
| 创新性 | 25% | 7.1–7.4 | 公式化模式、受控 Agent、补充通知确认、交互创新 |
| 前端工程质量 | 10% | 由 B 的前端说明/截图补充 | 本文不展开组件与性能细节 |
| 实用性 | 10% | 1.3、5.1–5.8、7.4 | 真实场景、与传统/通用 AI 对比 |
| 用户体验 | 10% | 7.3 + B 的界面截图/流程图 | 出处弹窗、差异预览、错误提示 |
| 社会价值与商业潜力 | 10% | 8.3 | 信息差、返工、B2C/B2B/API |